

Informe final — NovaTareas Pro

Universidad Gerardo Barrios — Módulo 4: Desarrollo de Aplicaciones con IA

Docente: Ing. Marco Arévalo Zambrano

Equipo 3: Saúl Oswaldo López Hernández (SMIS108421) · Moises Antonio Martínez (SMIS071221) · Enson Onan Carranza Rodríguez (SMIS013020)

Aplicación en línea	https://novatareas.polarzero.dev
Versión	1.0.0 (v1.0.0 → commit fa539b9)
Repositorio	https://github.com/Saul1hdz/NovaTareas
Estado del pipeline	En verde sobre el commit etiquetado
Fecha	19 de agosto de 2026

1. Resumen ejecutivo

NovaTareas Pro es un gestor de tareas con recomendaciones de inteligencia artificial, pensado para estudiantes universitarios salvadoreños. Se usa por tres vías —dashboard web, bot de Telegram y una API protegida— sobre una misma base de datos y una misma lógica.

El proyecto **está publicado y operando** con dominio y HTTPS propios. Lo que respalda esa afirmación, comprobable ahora mismo:

```
GET /api/v1/health/ready → 200 {"status":"ok","checks":{"database":true}}
GET /api/v1/metadata → 200 {"version":"1.0.0", ...}
```

En números: **202 pruebas automatizadas** en 30 archivos contra PostgreSQL 16 real, un pipeline que construye y **arranca** la imagen de producción antes de darla por buena, un evento de registro por cada solicitud con identificador de correlación, y una línea base de rendimiento medida con p50, p95 y tasa de error.

Lo que no está resuelto se declara en la sección 12, sin adornos.

2. El problema y para quién

Un estudiante universitario lleva a la vez varias asignaturas, proyectos de grupo y responsabilidades personales, todo compitiendo por la misma agenda. Una encuesta aplicada en febrero de 2026 a estudiantes de la UGB dio números claros:

- **80 %** considera difícil o muy difícil organizar sus tareas.
- **80 %** olvida al menos una entrega —hasta tres— por ciclo.
- Entre **80 % y 90 %** siente que esa desorganización le baja las notas.

Existen alternativas (Todoist, Notion, Trello), pero cobran, están pensadas en inglés y ninguna aprende del historial de quien las usa. Ese último punto es el que define este proyecto: **las recomendaciones se apoyan en las tareas que la persona ya cerró antes**, no en consejos genéricos.

3. La solución y dónde está la IA

Tres puertas, una sola lógica

Puerta	Para qué	Autenticación
Dashboard web	Gestión visual: prioridades, etiquetas, historial, comentarios, colaboración	Cookie de sesión firmada
Bot de Telegram	Crear tareas conversando, pedir consejo, recibir avisos y recordatorios	Vinculación con código de un solo uso
API /api/v1/	Exponer la capacidad inteligente a clientes autorizados	Bearer AI_API_KEY

La cascada de IA

La decisión de diseño con más consecuencias: **el servicio siempre responde**.

1. z.ai (glm-4.5-flash) ← mejor calidad
2. Ollama local ← si z.ai no responde o se agotó la cuota
3. Historial propio ← recuperación semántica sobre tareas archivadas
4. Reglas locales ← deterministas, sin red

Por eso /api/v1/health devuelve 200 aunque z.ai y Ollama estén caídos. La contrapartida honesta es que la calidad baja escalón a escalón, y por eso **cada recomendación guarda de qué fuente salió** (zai, ollama, history, rules): una caída de calidad se puede atribuir al escalón que la causó.

El usuario puede valorar cada recomendación con positiva o negativa, y esa valoración entra en el prompt siguiente.

Código: src/lib/aiEngine.js, src/lib/ai/providers.js, src/lib/rag.js.

4. Arquitectura

Monolito modular: una sola aplicación desplegable, separada por dentro en capas que no se mezclan. El middleware registra y filtra, las rutas traducen HTTP, el dominio tiene las reglas, la capa de IA no sabe de base de datos y la persistencia no aplica reglas de negocio.

Esa separación es la que permite exponer la misma capacidad como API externa sin arrastrar el modelo de usuarios detrás.

El detalle —diagrama por capas, responsabilidades, procesos desplegados y las tres condiciones concretas que justificarían pasar a microservicios— está en docs/ARQUITECTURA.md.

5. API y contratos

El servicio publica su propio contrato en /api/v1/metadata, así que un cliente no depende de documentación que pueda quedar desfasada.

Endpoint	Método	Autenticación
/api/v1/health	GET	Ninguna
/api/v1/health/ready	GET	Ninguna (sonda de despliegue: consulta la base)
/api/v1/metadata	GET	Ninguna
/api/v1/recommend	POST	Bearer AI_API_KEY

Entrada: titulo (obligatorio, máx. 200), descripcion (opcional, máx. 1000), prioridad, tipo_usuario, fecha_limite.

Salida: recomendacion, fuente y el eco de la tarea normalizada.

Las validaciones y los errores controlados tienen pruebas propias (tests/aiEngine.test.js, tests/api.test.js).

Contratos completos en api.md.

6. Datos

PostgreSQL 16 es el **motor único**: SQLite se retiró por completo y una prueba (tests/noSqliteDialect.test.js) impide que el dialecto vuelva a colarse.

- **17 tablas** agrupadas por propósito: identidad, tareas, colaboración, inteligencia, integraciones y defensa.
- **7 migraciones** versionadas con Drizzle, idempotentes y verificadas al reejecutarse.
- Tokens de Google **cifrados con AES-256-GCM**; el esquema rechaza guardarlos en claro.
- Los contadores de límites de intentos viven en la base, no en memoria: por eso sobreviven a un reinicio y servirían con varias instancias.

Diseño y diccionario de datos: docs/POSTGRESQL_DISENO_BLOQUE_2.md.

7. Calidad: pruebas y automatización

Pruebas

202 casos en 30 archivos, todos contra PostgreSQL 16 real, no contra un sustituto. Los endpoints se ejercitan importando el handler y pasándole un Request, sin levantar servidor. Ningún servicio externo se llama de verdad: z.ai va vacío y Ollama apunta a un puerto cerrado, así que la suite es determinista, no consume saldo y no falla por red.

Áreas cubiertas: autenticación y correo, autorización y colaboración, CSRF, cascada de IA y prompt, API pública, integraciones (Google, Telegram, cron), recordatorios, observabilidad, esquema y cifrado.

Entre ellas hay **diez casos adversariales explícitos**, cada uno con su prueba: mutación de origen cruzado con la cookie de la víctima, acceso a la tarea de otro usuario, un colaborador «lector» intentando editar, reúso de un token consumido, enumeración de cuentas por la respuesta de recuperación, límites bajo concurrencia, cron y webhook sin secreto, un ejecutable renombrado a .png, y un token de Google sin cifrar.

Pipeline

.github/workflows/ci.yml ejecuta en cada push: revisión de tipos → migración sobre PostgreSQL 16 efímero → comprobación transaccional → las 202 pruebas con cobertura → artefacto de cobertura → build → **construcción de la imagen de producción** → **arranque real de esa imagen** y curl contra /health/ready.

Es verificable públicamente: la ejecución sobre el commit etiquetado fa539b9 terminó en verde.

8. Observabilidad, rendimiento y escalabilidad

Qué se registra

Un evento JSON por solicitud, con identificador de correlación, ruta normalizada, estado, resultado, duración, coste de PostgreSQL y versión del código:

```
{ "ts": "2026-08-14T02:52:35.328Z", "level": "info", "event": "http_request",  
  "request_id": "a361aae4-...", "method": "GET", "route": "/api/tasks", "status": 200,  
  "outcome": "success", "duration_ms": 24, "app_version": "dev",  
  "db_queries": 2, "db_duration_ms": 18.93 }
```

El mismo request_id viaja al cliente en la cabecera x-request-id, así que un fallo reportado se localiza en el log sin buscar por hora.

Los campos publicables son una **lista blanca**: nunca salen cuerpos, cabeceras, contraseñas, correos ni el *query string*, y las rutas se normalizan (numérico → :id, cadena larga → :token) para que un enlace de invitación no acabe registrado. Lo verifica tests/observability.test.js.

Línea base medida

Escenario: GET /api/tasks con sesión iniciada, 30 solicitudes secuenciales, 3 de calentamiento, sobre Docker Desktop en Windows 11.

Escenario	p50	p95	Máx	Error	req/s
tasks c=1	16,19 ms	23,74 ms	49,39 ms	0 %	56,4
tasks c=4	31,36 ms	87,60 ms	109,83 ms	0 %	104,4
tasks c=8	73,37 ms	208,76 ms	227,88 ms	0 %	93,0
tasks c=16	172,05 ms	291,81 ms	546,09 ms	0 %	71,8

Tasa de error 0 % en todas las tandas.

Cuello de botella

La instrumentación permite comparar lo que mide el cliente con lo que el servidor cree haber tardado, y ahí aparece el hallazgo:

Tramo	Tiempo	Porcentaje
Fuera de la aplicación (red de Docker Desktop en Windows)	~10,9 ms	73 %
Aplicación sin base de datos	~1,0 ms	7 %
PostgreSQL (2 consultas)	~3,0 ms	20 %

El cuello de botella de la latencia percibida no está en el código. Casi tres cuartas partes se consumen antes de que la solicitud llegue al proceso Node. Sin la instrumentación, la conclusión natural habría sido optimizar el código equivocado.

Mejora aplicada

Servir el proyecto compilado en lugar del servidor de desarrollo:

Métrica (tasks c=1)	astro dev	Compilado	Cambio
p50	20,67 ms	16,19 ms	-21,7 %
Throughput	47,9 req/s	56,4 req/s	+17,7 %

Confirmado en dos ejecuciones independientes (-21 % y -22 %), así que no es ruido. No mejora el techo bajo carga, lo cual es coherente con el diagnóstico: la saturación es del proceso, no de la compilación.

Se evaluó y **se descartó** cachear la validación de sesión: ahorraría ~1,5 ms sobre 15 ms y retrasaría la invalidación inmediata de sesiones al cambiar la contraseña. La evidencia no justificaba el coste de seguridad.

Plan de escalabilidad

Un proceso Node satura entre 65 y 105 req/s y la latencia crece linealmente a partir de 4 peticiones concurrentes. Cada palanca tiene un indicador que dice **cuándo** tirar de ella:

Recurso	Cuándo	Indicador
Más instancias de web	Antes que nada, si sube la concurrencia	duration_ms p95 > 300 ms con CPU alta
Caché de recomendaciones	Al repetirse peticiones de IA equivalentes	ai_duration_ms domina duration_ms
Cola o *workers*	Si la IA pasa a ser síncrona y masiva	ai_duration_ms p95 > 5 s sostenido
Réplica de lectura	Cuando la base supere el 20 % del tiempo	db_duration_ms > 50 % de duration_ms

Análisis completo y datos crudos: docs/OBSERVABILIDAD_SEMANA_5.md y los 13 archivos de docs/mediciones/.

9. Seguridad

Controles activos, cada uno con la prueba que lo respalda:

Ámbito	Control
Sesión	JWT firmado con versión y vencimiento; cookie HttpOnly, SameSite=Lax, Secure bajo HTTPS. Cambiar la contraseña invalida las sesiones anteriores
Cuentas	Confirmación de correo antes del primer login; tokens de un solo uso; la recuperación responde igual exista o no la cuenta
Origen cruzado	Middleware único que falla cerrado y respeta X-Forwarded-Proto
Autorización	Ownership en toda consulta; tres niveles de colaboración; solo el propietario borra o vuelve privada la tarea
Abuso	Límites persistidos en PostgreSQL, verificados incluso bajo concurrencia
Datos	Tokens de Google cifrados; avatares validados por contenido, no por el tipo declarado
Integraciones	Webhook y cron con secreto comparado en tiempo constante; state de OAuth firmado
Registros	Lista blanca de campos: no salen cuerpos, cabeceras, contraseñas ni correos

Un detalle que vale la pena señalar porque salió de un incidente real: la comprobación de origen cruzado fallaba en producción detrás de Cloudflare, porque el servidor se veía a sí mismo en `http` mientras el navegador hablaba `https`. Se corrigió respetando `X-Forwarded-Proto`, y quedó cubierto con `tests/csrf.test.js`.

Modelo de amenazas, controles y **siete límites declarados fuera de alcance**: `docs/SEGURIDAD.md`.

10. Release, verificación y vuelta atrás

- **Versión 1.0.0**, declarada de forma coherente en el código (`AI_META`), en el servicio en vivo (`/api/v1/metadata`), en el README y en el manifiesto.
- **Etiqueta** `v1.0.0` → **commit** `fa539b9`, que contiene tanto el manifiesto como las notas que lo describen.
- `release-manifest.yml`: qué se publicó exactamente — código, contrato, modelo, prompt, migraciones, imagen, ambiente y conjunto de pruebas.
- `CHANGELOG.md`: notas de la versión y limitaciones.
- **Prueba de humo**: automática en CI contra la imagen real, y manual con las siete comprobaciones del runbook tras cada publicación.
- **Vuelta atrás**: `git checkout v1.0.0` y reconstruir, con una advertencia que conviene no aprender por las malas — **volver el código no revierte el esquema**; si hubo migraciones, primero se restaura la copia de `pg_dump`.

11. Riesgos iniciales y cómo cerraron

Riesgo identificado al inicio	Estado	Evidencia
SQLite temporal, había que migrar	[OK] Cerrado	PostgreSQL 16 único; prueba que impide el retroceso
Límite de intentos en memoria	[OK] Cerrado	Tabla <code>rate_limit_hits</code> , verificada bajo concurrencia
Dependencias con avisos de seguridad	[OK] Cerrado	<code>npm audit</code> sin vulnerabilidades
Recuperación por preguntas de seguridad	[OK] Cerrado	Sustituida por correo verificado
Bloques grandes de <code>innerHTML</code>	[OK] Cerrado	Ninguno queda en el dashboard
Cobertura incompleta	[Parcial] Casi	Faltan la conversación del bot y el RAG
Netcup sin preparar	[OK] Cerrado	Publicado con HTTPS y sonda en verde
Dashboard monolítico	[Pendiente] Abierto	Casi 4.000 líneas y <code>onClick</code> en el marcado

12. Limitaciones conocidas

1. **La publicación es manual**. El pipeline construye y arranca la imagen, pero no despliega.
2. **Una sola instancia de bot**: usa polling, y dos procesos con el mismo token se roban los mensajes. La web sí escala.
3. **Sin segundo factor**: una contraseña comprometida da acceso completo.
4. **Las tareas no están cifradas en reposo**; solo los tokens de Google.

5. **La confirmación de cuenta depende del SMTP:** si no responde, nadie se registra ni recupera contraseña.
6. **La cobertura se mide pero no se exige:** no hay umbral que rompa el pipeline.
7. **Huecos de prueba:** conversación del bot, RAG de extremo a extremo y flujo visual del dashboard.
8. **El RAG está conectado a medias:** los vectores existen, no todas las rutas los consumen.
9. **Google Calendar a medio camino:** OAuth y tokens probados, falta conectarlo al dashboard.
10. `dashboard.astro` **supera las 3.900 líneas** con CSS y JavaScript dentro: es la deuda que más estorba.
11. **Una prueba inestable conocida** (`tests/telegramSessions.test.js`), por desfase de reloj entre el contenedor y el host. Pasa al repetir.

13. Sigüientes pasos

1. **Despliegue automático** en el pipeline: es el último tramo que falta.
2. **Partir** `dashboard.astro`, empezando por el JavaScript de cliente.
3. **Conectar Google Calendar** al dashboard.
4. **Agregar las valoraciones positiva o negativa en un informe** que muestre si la calidad sube o baja: los datos ya se están guardando.
5. **Terminar de conectar el RAG** en todas las rutas.
6. **Asignación de tareas entre miembros** y aviso por Telegram al modificarlas.
7. **Versionar el prompt del sistema** más allá de la etiqueta actual, para atribuir cambios de calidad a cambios concretos.

14. Dónde está cada cosa

Necesito...	Documento
Entender el sistema completo	<code>README.md</code>
Capas y decisiones de diseño	<code>docs/ARQUITECTURA.md</code>
Controles de seguridad y su alcance	<code>docs/SEGURIDAD.md</code>
Instrumentación, medición y escalabilidad	<code>docs/OBSERVABILIDAD_SEMANA_5.md</code>
Operar el servidor	<code>docs/DESPLIEGUE.md</code>
Permisos del trabajo en equipo	<code>docs/MODO_COLABORATIVO.md</code>
Contratos de la API	<code>api.md</code>
Qué se publicó en la 1.0.0	<code>release-manifest.yml</code>
Notas de versión	<code>CHANGELOG.md</code>
Índice de la documentación	<code>docs/README.md</code>

Datos crudos de las mediciones: `docs/mediciones/`. Evidencias de entrega: `docs/Evidencia_Semana3.pdf` y `docs/Semana5_Observabilidad_Rendimiento_NovaTareas.pdf`.

15. Cierre

El proyecto arrancó como una aplicación local sobre SQLite, sin pruebas, sin integración continua y sin forma de medir nada. Termina publicado con dominio y HTTPS propios, sobre PostgreSQL 16, con 202 pruebas automatizadas, un pipeline que verifica la imagen que se despliega, instrumentación que permitió encontrar el cuello de botella real —que no estaba donde parecía— y una versión etiquetada a la que se puede volver.